IN2013 — Object-Oriented Analysis & Design

2025/26 Coursework Assignment 2 (Design)

Campus Booking System (CBS)

Version 6.0 — generated 2025-10-31

Dr. Ali Sarvghad

Q1 — Design Class Diagram [40 marks]

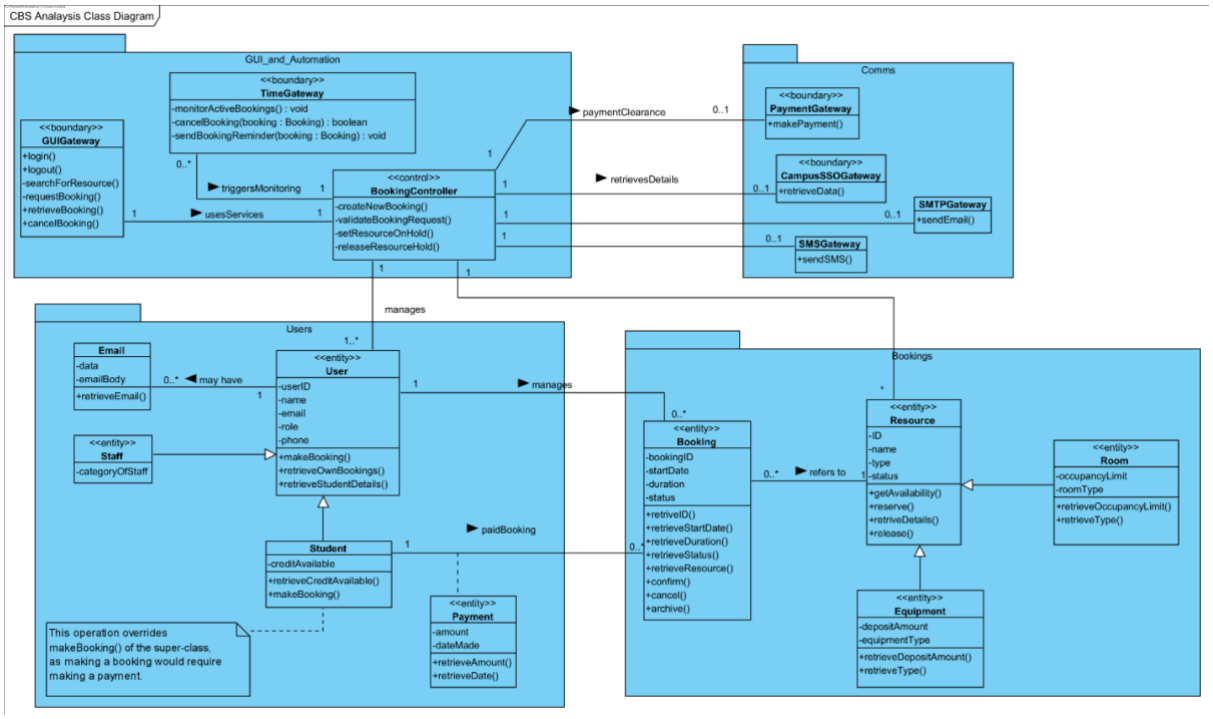


Figure 1 CBS-Analysis class diagram.

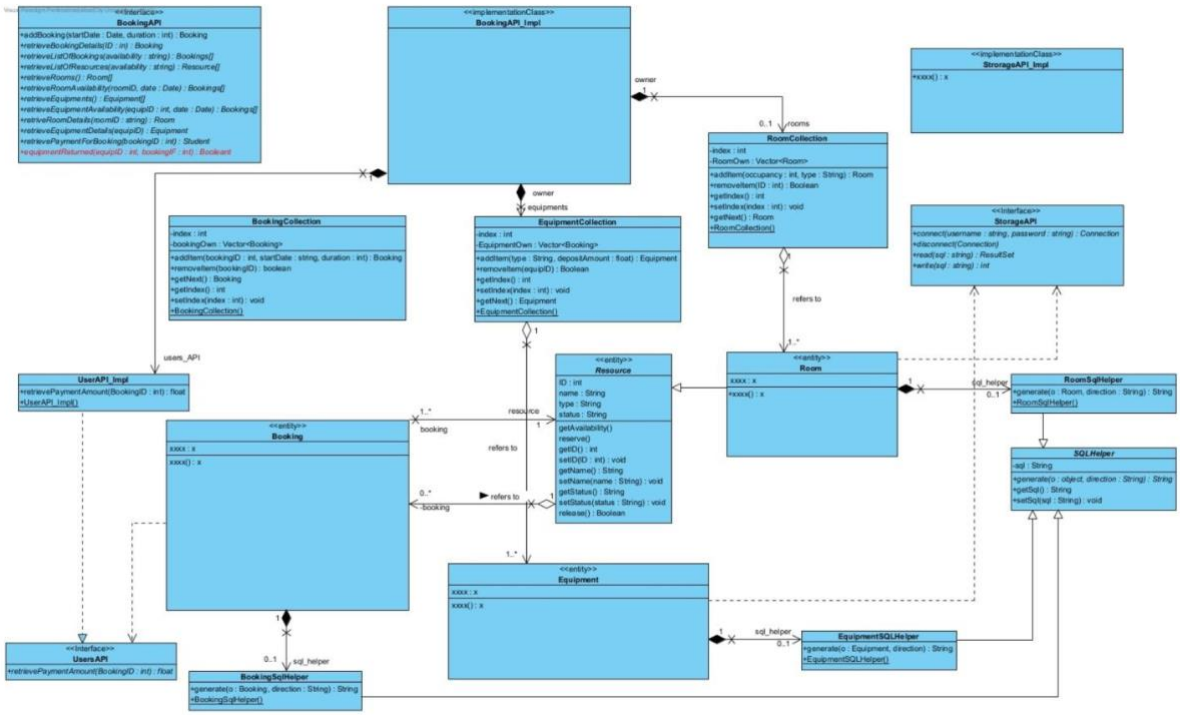


Figure 2 this figure shows Design-level implementation of CBS Booking subsystem.

Figure 1 shows the analysis class diagram of CBS system. Figure 2 shows design-level class

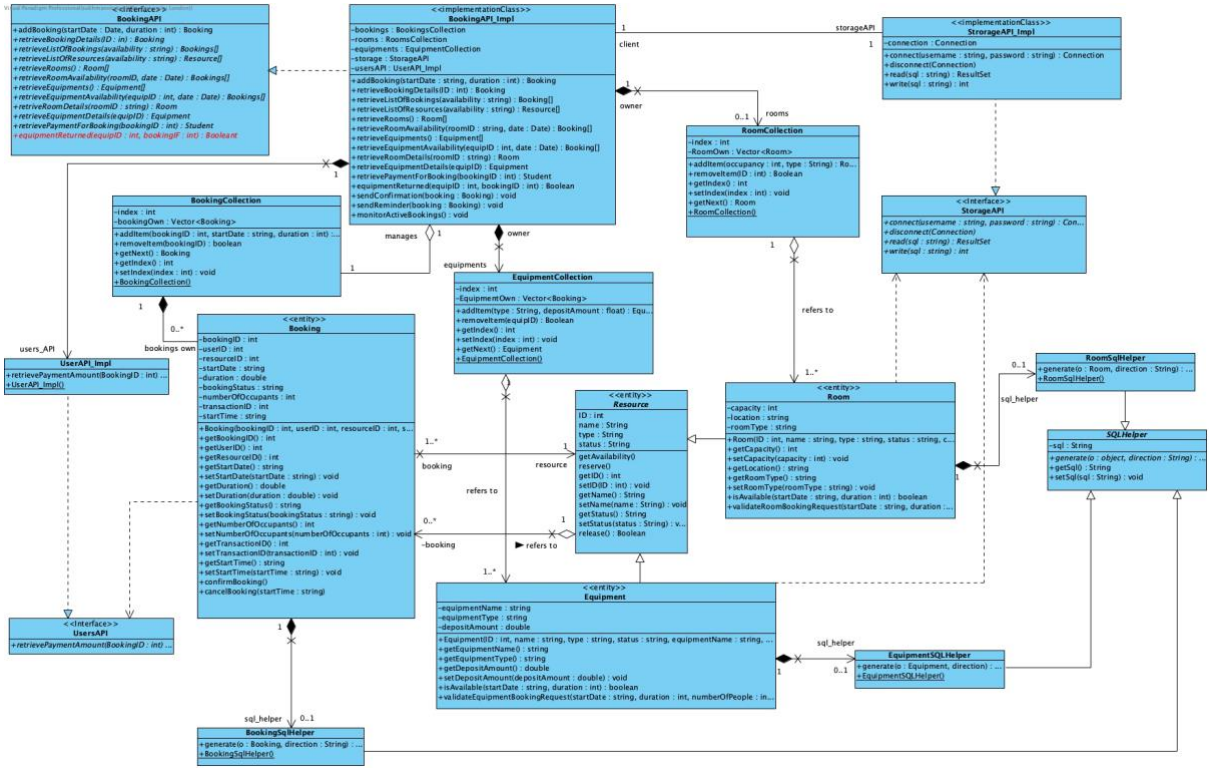
diagram of the subsystem Booking with some details omitted. Answer the following

questions using Figure 1 plus the CBS scenario:

1. Complete the list of attributes and operations for classes:
    - Booking
    - Booking API_Impl
    - Room
    - Equipment
    - StorageAPI_impl

You should include “design” level of details for attributes and operations, including visibility, scope, type, initial value, and multiplicity for attributes and visibility, scope, and complete signature for operations. Add constructors, getters, and setters to classes as needed. The xxxx:x and xxxx():x are only indicators of missing attributes and operations not actual attributes or operations.

Complete Diagram:

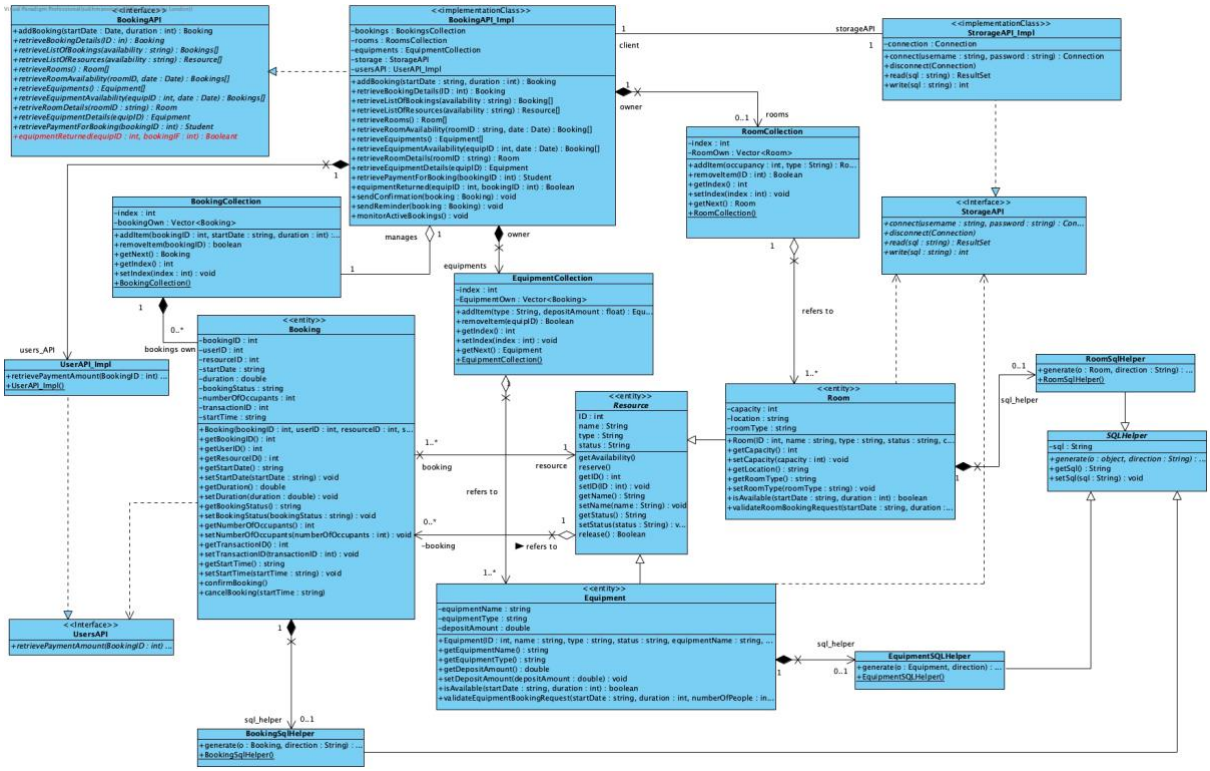


<u>NOTE: Some attributes didn't have setters to reinforce immutable objects.</u>

2. Identify the correct relationship (e.g., composition, aggregation) between the following pairs of classes/interfaces. Also, identify correct multiplicity and navigability (where applicable):

- **StorageAPI -- StorageAPI_Impl**
- **BookingAPI_Impl -- BookingAPI**
- **BookingAPI_Impl -- BookingCollection**
- **BookingAPI_Impl -- StorageAPI_Impl**
- **Resource -- Equipment**
- **BookingCollection -- Booking**
- **Booking -- StorageAPI**
- **BookingSqlHelper – SQLHelper**

→Please note that the relationships between these classes are not shown in current version of Figure 2.



2. What is the scope of attribute sql in SQLHelper? How sql can be accessed in subclasses of SQLHelper? Explain your answer briefly.

- The attribute 'sql' in 'SQLHelper' class with a string return type has a private scope.
- The subclasses of 'SQLHelper', which are 'BookingSqlHelper, RoomSqlHelper, EquipmentSQLHelper' cannot directly access 'sql' attribute from 'SQLHelper'. These subclasses will need to use the public methods within 'SQLHelper'. The first is getSql(), which will return the SQL String and the other setSql(String sql) which will set the SQL string.
- Having the private scope for the attribute 'sql' provides encapsulation (which is an OOP Principle). This allows for the superclasses to control the subclasses which access the attribute and to use a consistent interface.
- If the 'sql' attribute was protected, this would lead to the subclasses being able to access the attribute directly, which can provide faster access times. However, this will reduce the maintainability of the attribute 'sql'.

3. What is the type of class Resource? Does it require a constructor? Explain your answer briefly.

- The class type of 'Resource' is an abstract class, and this class can have a protected constructor to initialise all the attributes which are 'ID, name, type, status'. All the attributes within 'Resource' are inherited by the subclasses of 'Resource', which are 'Room' and 'Equipment', and having the constructor protected can prevent external classes from instantiating 'Resource'. Also, having a protected constructor can allow for the subclasses of 'Resource' to call the protected constructor by using the keyword 'super()', which can be used to initialise inherited attributes.

4. Define correct visibility for all the attributes and operations in class Resource. Explain your answer briefly.

Attributes	***Type***
#ID	*Protected*
#name	*Protected*
#type	*Protected*
#status	*Protected*

Operations	***Type***
#Resource (ID : int, name : string, type : string, status : string)	*Protected (Constructor)*
+getAvailability(): Boolean	*Public*
+reserve() : void	*Public*
+release() : Boolean	*Public*
+getID() : int	*Public*
+setID(ID : int) : void	*Public*
+getName() : string	*Public*
+setName(name : string) : void	*Public*
+getStatus() : string	*Public*
+setStatus(status : string) : void	*Public*

- The attributes are protected and not private because the subclasses will need to be able to access those attributes directly for the operations which they can

conduct. An example of this can be 'this.name', which can be used to access a particular name.

- The reason why the methods are public and not private is because the 'Resource' entity allows for the subclasses to have a valid access to the protected attributes which were declared and to provide an interface for those subclasses which can help them interact with 'Resource'.

5. Why the +equipmentReturned() operation is needed in interface BookingAPI? Should processing this method include releasing the deposit made by a student? Explain your answer briefly.

- The 'equipmentReturned()' operation is necessary because the scenario states that 'the users will pick up and drop off rented equipment according to the booking time'. However, this operation will not be needed for the rooms since the rooms will automatically become available once the duration of the booking is over.
- Having the 'equipmentReturned()' operation in place means that an end-user of the system will need to trigger this operation to verify that the equipment which has been rented has been successfully returned, which in this case the end-user will be the staff.
- Processing the 'equipmentReturned()' method should not include or trigger a deposit release for the deposit made by a student. This should be handled separately on the payment gateway. This is because the scenario states that 'The staff in charge of handling/receiving equipment will release the deposit by using transaction ID'. The 'equipmentReturned()' function will only be responsible for showing on the system whether if the equipment (laptops, tablet, VR, headsets, ballet shoes, tennis rackets/tennis balls) were returned by the student. The 'equipmentReturned()' will need to be separate from the deposit release, as the deposit release will be done on the payment gateway.

Q2-State machine diagram [30 marks]

Figure 3 shows the behavioural state machine diagram for the class Booking. Complete the figure by filling the "?" for each state, event and transition. There is a total of 16 question marks. To answer Q2, submit the diagram with question marks (?) replaced by your answers.

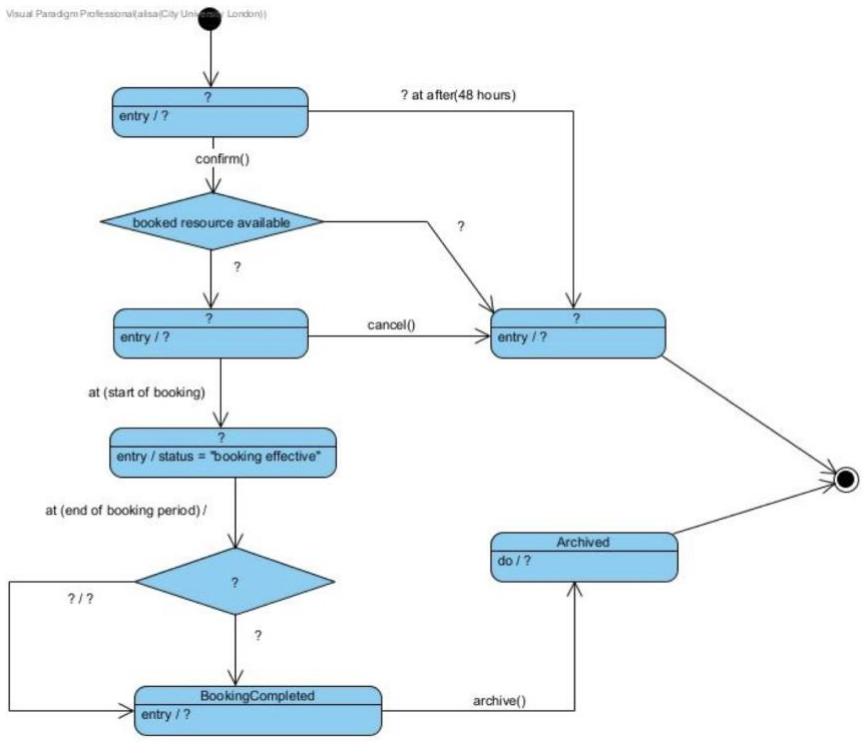
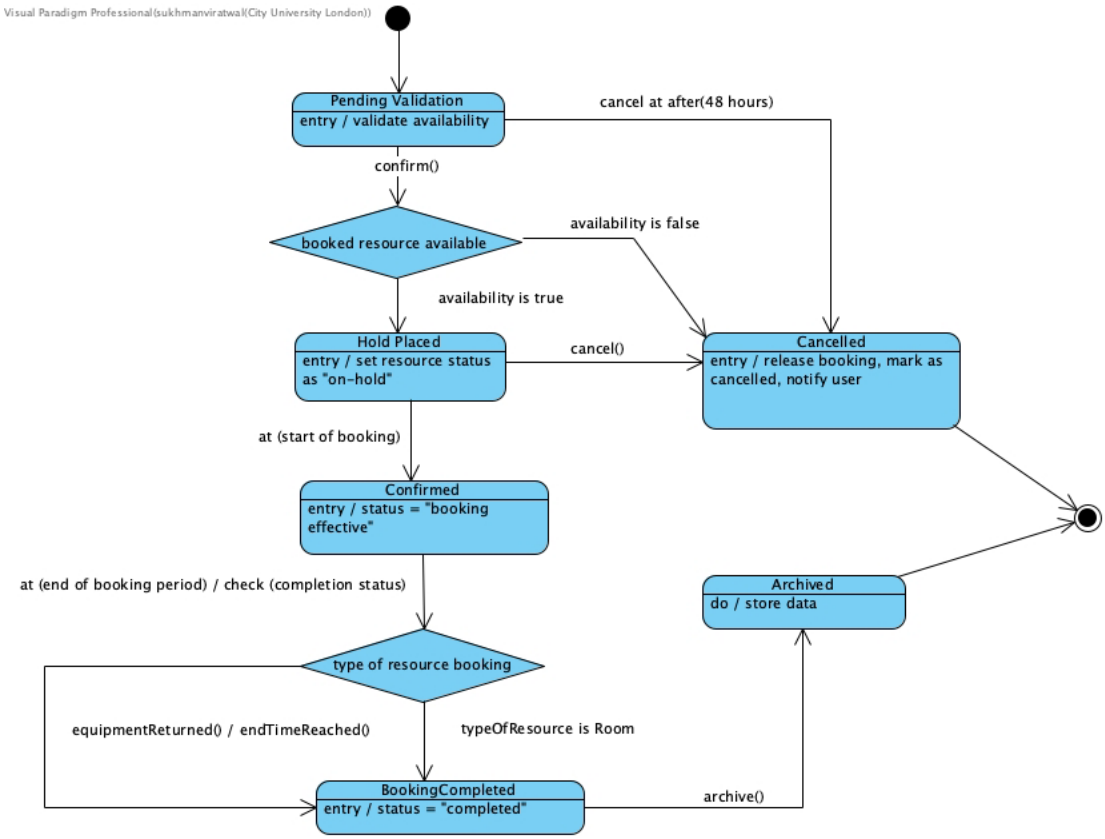


Figure 3 Incomplete behavioural state machine for Booking class.

Complete Diagram:



Q3- Deployment diagram [15 marks]

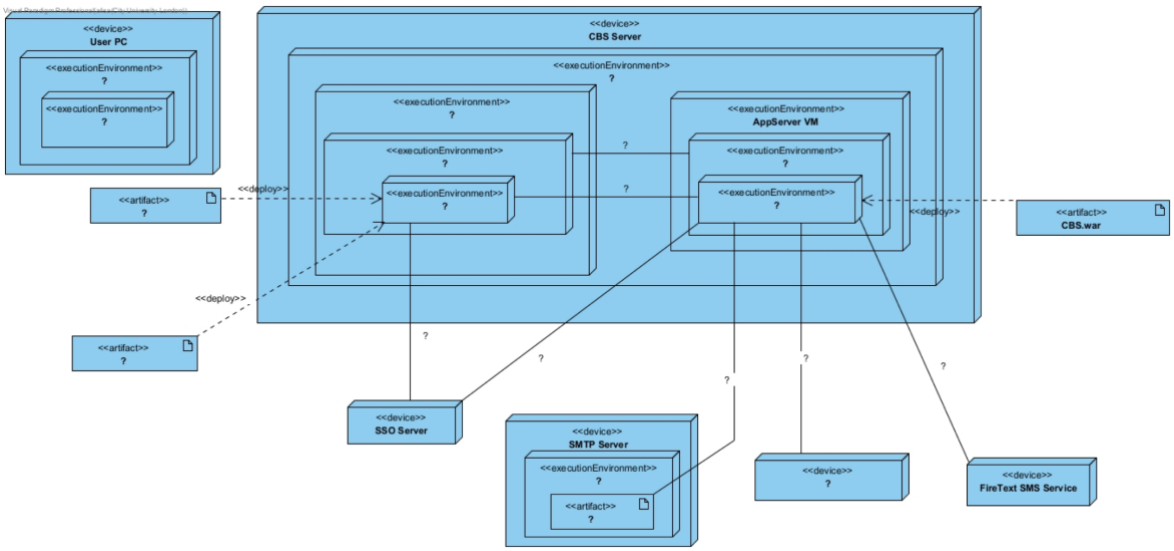


Figure 4 Incomplete Deployment Diagram of CBS

Figure 4 presents a partial Deployment Diagram of CBS. Use the following information to complete all places marked with a question mark (?). There is a total of 20 question marks. To answer Q3, submit the diagram with question marks (?) replaced by your answers.

Runtime infrastructure:

- User PC: Windows 11 with a Web Browser.
- CBS Server (physical host) running VMware ESXi 7.0, which contains two VMs:

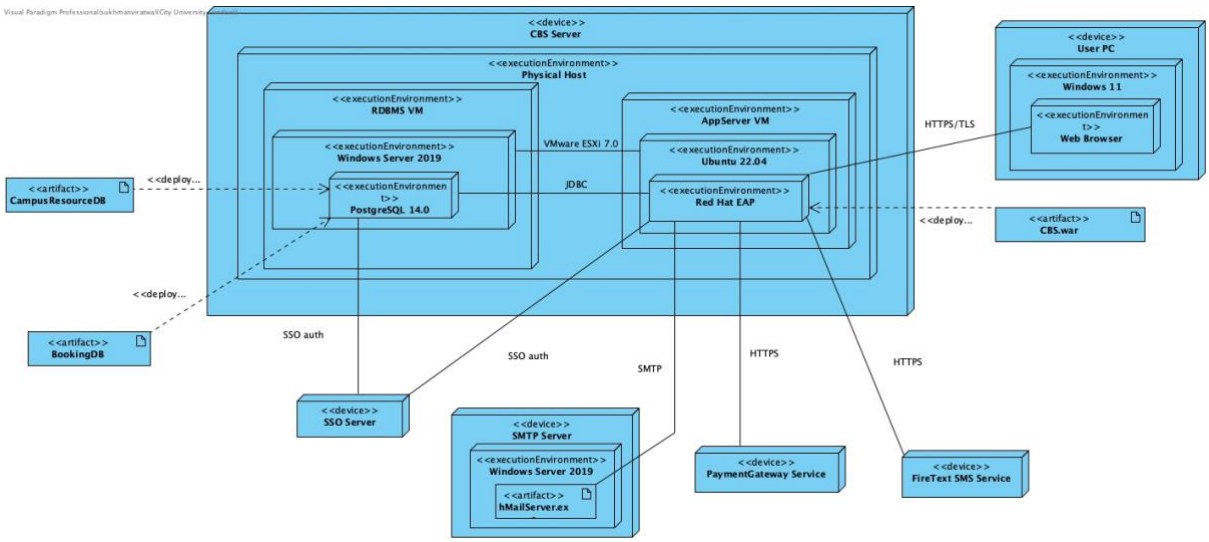
1) AppServer VM → Ubuntu 22.04 → Red Hat EAP application server. The CBS web app is packaged as an artifact CBS.war and is deployed here.
2) RDBMS VM → Windows Server 2019 → PostgreSQL 14.0. Two database artifacts exist: BookingDB and CampusResourceDB (treat them as artifacts deployed to PostgreSQL).

- SMTP Server: Windows Server 2019 running hMailServer.exe.
- SSO Server: campus single-sign-on provider (no further internals required).
- PaymentGateway Service: external provider.
- FireText SMS Service: external provider.

Required communication links (label each with the protocol):

- Browser ↔ App Server: **HTTPS/TLS**.
- App Server ↔ PostgreSQL: **JDBC** over the **PostgreSQL protocol** (assume TLS is
- enabled).
- App Server → SMTP Server: **SMTP**.
- App Server ↔ SSO Server: **SSO authentication** (SAML 2.0 or OpenID Connect over HTTPS—label as **SSO auth**).
- App Server → PaymentGateway: **HTTPS (PaymentGateway API).**
- App Server → FireText: **HTTPS (FireText REST API)**.

<u>Complete Diagram:</u>



NOTE: ***Q3 stated to add a required communication link between "Browser ←→ App Server: HTTPS/TLS", but there was no '??' label. So, I've added a required communication in case the communication was accidentally missed out.***

Q4-Testing [15 marks]

Test Number: TC01 (Successful Room Booking)

Objective:

To test and validate whether if the room booking is successfully completed when there is a valid number of occupants and the room is available.

Set Up:

- System is operational.
- The user is logged onto the system using their university credentials.
- A room called 'A101' exists within the database, ready to be booked.
- Room 'A101' has maximum number of occupants at '10', where the status is 'available'.
- There are no other bookings for the room 'A101' at the student's wanted start date/time.

Expected Results:

1) The system places the status of the room booking 'on hold'.
2) The system checks if the number of occupants is within the correct limit.
3) Moreover, the system will check the booking date/time and checks if the room is available at the requested date/time.
4) All checks are successful and the booking status is 'confirmed'.
5) A new booking record will be created within the booking database.

- bookingID = 001
- userID = 0123
- resourceID = 0101
- startDate = 28/12/2025
- startTime = 10:00
- duration = 120
- numberOfOccupants = 6
- bookingStatus = "Confirmed"

6) System sends the user a booking confirmation via email.

Test:

1) The user logs onto the system with their university credentials.
2) A user will request to book a specific room (e.g., A101).
3) The user will provide details such as the booking start date/time and the duration. The start date and time will be '28/12/2025' at '10:00' respectively.
4) The user will also provide the number of occupants for the room, which doesn't exceed the maximum limit. The number of occupants the user will provide for the room is '6'.

Test Record: • A new booking record for room 'A101' is created within the booking database. • The booking status changes 'confirmed' for the requested start date/time.	
Date: 27/12/2025	**Tester:** John Smith
Result: PASS	

Test Number: TC02 (Successful Equipment Booking)
Objective: To test and validate whether if the equipment booking is successfully completed when a valid booking start time/date and duration is provided, as well as a successful deposit by the student.
Set Up: • System is operational. • The user will log onto the system using their university credentials. • Equipment with a name 'VR105' exists within the booking database, where the status is 'available'. • There are no other bookings for 'VR105' at the wanted start date/time.
Expected Result: 1) The system places the equipment (e.g., VR105) 'on hold'. 2) The system conducts check to see if the requested equipment is available at the start date/time provided by the user. 3) A booking is created within the database. 4) A deposit of '£100' is taken for the equipment from the user's payment method. 5) The booking status of the equipment is set to 'confirmed'. 6) A new booking record will be created for the equipment 'VR105' within the booking database. • bookingID = 001 • userID = 0123 • resourceID = 0105 • startDate = 31/12/2025 • startTime = 12:00 • duration = 60 (minutes) • deposit = 100

- bookingStatus = “Confirmed”

7) The user receives a confirmation email for the booking made.

Test:

1) The user logs onto the system using their university credentials.
2) User requests a booking for a particular equipment (e.g., VR105).
3) The user will provide appropriate booking details such as start date/time and duration. The start date and time will be ‘31/12/2025’ at ‘12:00’ respectively.
4) The user will also put in the correct amount of deposit, which is ‘£100’.

Test Record:

- A new booking record is created for the equipment ‘VR105’ within the booking database.
- The booking status for the equipment ‘VR105 changes to ‘confirmed’.
- A record is also created for the deposit which was provided by the student for the equipment ‘VR105

Date: 27/12/2025	**Tester:** John Smith

Result:

PASS

Test Number: TC03 (Room ‘Number of Occupants’ Exceeds Limit)

Objective:

To test and validate that the system will not accept a room booking since the number of occupants of the room exceeds the maximum limit.

Set Up:

- System is operational.
- The user logs onto the system using their university credentials.
- The room called ‘A101’ exists within the bookings database.
- The room ‘A101’s maximum number of occupants are ‘5’ and the status is ‘available.’

Expected Results:

1) The system writes the status of the room booking from ‘available’ to ‘on hold’.
2) The system checks the details such as the start date/time to see if the room is available at the requested start date/time.
3) The system also checks the input value for the number of occupants and sees if the value exceeds the maximum limit or not.

4) The number of occupants value provided by the user exceeds the maximum limit, the validation fails. 5) The system displays an error message stating the reason why the room booking couldn't be completed (since the room booking's number of occupants exceeds the limit). 6) System cancels the room booking. 7) The status of the room booking changes from 'on hold' back to 'available' by the system.
Test: 1) The user will log onto the system using their university credentials. 2) The user will request a booking for a room (e.g., A101). 3) The user will provide a start time/date, duration as well as the number of occupants for the room. The start date and time will be '29/12/2025' at '16:00' respectively. 4) The input value from the user for the number of occupants for the room will exceed the maximum limit. The input value for number of occupants from the user will be '8'.
Test Record: • No booking record will be created within the booking database for the room 'A101'. • The room 'A101' status will remain as 'available.'
Date: 27/12/2025 | **Tester:** John Smith
Result: PASS

Test Number: TC04 (Room 'Start Date/Time' Already Booked)
Objective: To test and validate that the system will not accept a room booking since the start date/time requested by the user is already reserved.
Set Up: • System is operational. • The user is logged onto the system using their university credentials. • A room called 'A202' exists within the database. • The room 'A202' already has a booking confirmed for the wanted start date/time, which will be on the '30/12/2025' at '13:00'.
Expected Result:

1) The system changes the status of the room booking from ‘available’ to ‘on hold’.
2) The system checks the details provided by the user, such as the start date/time, duration and number of occupants to check if those fields are valid.
3) The start date/time for a particular room is the same with another booking made for the same room and start date/time, the validation fails.
4) The system displays an error message stating the reason why the room booking has failed (which in this case is because the same room is already booked for the same start date/time provided by the user).
5) The system cancels the booking
6) The status of the room booking changes from ‘on hold’ to ‘available’.

Test:

1) The user will log onto the system using their university credentials.
2) The user will request a booking for a room ‘A202’.
3) After that, the user will provide the booking start date/time, duration as well as the number of occupants. The start date/time will be ‘30/12/2025’ at ‘13:00’ respectively.
4) The user will make sure the start date/time provided to the system is already reserved.

Test Record:

- No new booking record will be created for the room ‘A202’ within the booking database.

Date: 27/12/2025	**Tester:** John Smith

Result:

PASS